

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №8

по дисциплине: "Логика и основы алгоритмизации в инженерных задачах"

на тему: " Обход графа в ширину"

Выполнил:

Студент группы 24ВВВ3

Макунькин А.М.

Мелюшев М.М.

Принял:

к.н., доцент, Юрова О. В.

к.т.н., Деев М.В.

Пенза 2025

Цель

Изучение обхода графов в ширину.

Теоретическая часть

Обход графа в ширину – еще один распространенный способ обхода графов.

Основная идея такого обхода состоит в том, чтобы посещать вершины по уровням удаленности от исходной вершины. Удалённость в данном случае понимается как количество ребер, по которым необходимо прейти до достижения вершины.

Ключевые принципы:

Выбор начальной вершины: Алгоритм начинает работу с произвольно выбранной вершины.

Помеченные посещенных вершин: чтобы избежать заикливания и повторного посещения одной и той же вершины, каждая посещенная вершина помечается специальным образом (например, заносится в массив visited).

Использование очереди: Для управления порядком обхода используется очередь. Вершины, находящиеся на одном уровне удаленности, обрабатываются раньше вершин следующего уровня.

Обход по уровням: Сначала обрабатываются все соседи начальной вершины (уровень 1), затем соседи соседей (уровень 2) и т.д.

Для несвязных графов: Если граф несвязный, алгоритм необходимо запустить повторно для каждой не посещённой вершины.

Псевдокод

Вход: G – матрица смежности графа.

Выход: номера вершин в порядке их прохождения на экране.

Алгоритм ПОШ

- 1.1. для всех i положим $NUM[i] = \text{False}$ пометим как "не посещенную";
- 1.2. **ПОКА** существует "новая" вершина v
- 1.3. **ВЫПОЛНЯТЬ** BFS (v).

Алгоритм BFS(v):

- 2.1. Создать пустую очередь $Q = \{\}$;
- 2.2. Поместить v в очередь $Q.push(v)$;
- 2.3. пометить v как "посещенную" $NUM[v] = \text{True}$;
- 2.4. **ПОКА** $Q \neq \emptyset$ очередь не пуста **ВЫПОЛНЯТЬ**

- 2.5. $v = Q.front()$ установить текущую вершину;
- 2.6. Удалить первый элемент из очереди $Q.pop()$;
- 2.7. вывести на экран v ;
- 2.8. **ДЛЯ** $i = 1$ **ДО** $size_G$ **ВЫПОЛНЯТЬ**
- 2.9. **ЕСЛИ** $G(v,i) == 1$ **И** $NUM[i] == False$
- 2.10. **ТО**
- 2.11. Поместить i в очередь $Q.push(i)$;
- 2.12. пометить v как "посещенную" $NUM[v] = True$;

Лабораторное задание:

- a. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G . Выведите матрицу на экран.
- b. Для сгенерированного графа осуществите процедуру обхода в ширину, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс `queue` из стандартной библиотеки $C++$.

Результат программы

```

Консоль отладки Microsoft Visual Studio
Input number of vertices : 10
0 1 1 0 0 1 0 0 0 0
1 0 0 1 1 1 1 1 1 1
1 0 0 0 1 0 1 0 0 1
0 1 0 0 0 0 1 0 0 1
0 1 1 0 0 1 0 1 0 1
1 1 0 0 1 0 0 1 1 1
0 1 1 1 0 0 0 0 1 1
0 1 0 0 1 1 0 0 0 1
0 1 0 0 0 1 1 0 0 1
0 1 1 1 1 1 1 1 1 0
Input start vert : 4
Path: 4 1 2 5 7 9 0 3 6 8

D:\pr\Lab8L\x64\Debug\Lab8L.exe (процесс 27732) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" -> "Параметры" -> "Отладка" -> "Автоматически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно:

```

Рисунок 1 – обход графа

Листинг

Файл 1-Lab3L.c

```
#define _CRT_SECURE_NO_WARNINGS

#include <iostream>
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#include <ctime>
#include <queue>

using namespace std;

void BFS(int** G, int numG, int* visited, int s) {
    queue<int> q;
    int v;
    visited[s] = 1;
    q.push(s);

    while (!q.empty()) {

        v = q.front();
        q.pop();
        printf("%3d", v);

        for (int i = 0; i < numG; i++) {
            if (G[v][i] == 1 && visited[i] == 0) {
                q.push(i);
                visited[i] = 1;
            }
        }
    }
}

int main() {

    int** G;
    int* visited;
    int numG, current;

    printf("Input number of vertission : ");
    scanf("%d", &numG);

    visited = (int*)malloc(numG * sizeof(int));
    G = (int**)malloc(numG * sizeof(int*));
    for (int i = 0; i < numG; i++) {
        G[i] = (int*)malloc(numG * sizeof(int));
    }

    for (int i = 0; i < numG; i++) {

        visited[i] = 0;
        for (int j = i; j < numG; j++) {
            G[i][j] = G[j][i] = (i == j ? 0 : rand() % 2);
        }
    }

    for (int i = 0; i < numG; i++) {

        for (int j = 0; j < numG; j++) {
            printf("%3d", G[i][j]);
        }
        printf("\n");
    }

    printf("Input start vert : ");
```

```
scanf("%d", &current);

printf("Path: ");

BFS(G, numG, visited, current);
printf("\n\n");

for (int i = 0; i < numG; i++)
    free(G[i]);
free(G);
free(visited);

return 0;
}
```

Вывод

В ходе выполнения лабораторной работы были разработаны программы для выполнения заданий Лабораторной работы №8. В процессе выполнения работы были использованы знания о обходе графа в ширину.

Ссылка на удаленный репозиторий GitHub:

<https://github.com/LareklAREK/LiOAiIZ-2OURSE-.git>

<https://github.com/Motorrr/LiOAvIZ.git>